GCP環境構築手順書

> 対象：開発メンバー 関連Issue：#2 GCPプロジェクト・環境構築

1. 概要

本ドキュメントでは、ローカル開発環境からVertex AI経由でGeminiを利用するための手順を説明する。

使用技術

項目	設定
Cloud	Google Cloud
AIサービス	Vertex AI
モデル	gemini-3.5-flash
リージョン	asia-northeast1
認証	Application Default Credentials (ADC)
Python	3.12
SDK	google-genai

2. 事前準備

以下を用意する。

- Google Cloudプロジェクトへのアクセス権
- Google Cloud CLI
- Python 3.12
- Git
- リポジトリのローカル環境

3. Google Cloud CLIの認証

3.1 CLIの確認

`gcloud --version` を実行する。

3.2 Googleアカウントの認証

`gcloud init` を実行し、対象のGoogleアカウントとGCPプロジェクトを選択する。

3.3 プロジェクトの確認

`gcloud config get-value project` を実行し、対象プロジェクトが選択されていることを確認する。

3.4 ADC認証

以下を実行し、ブラウザで認証を完了する。

```
gcloud auth application-default login
```

認証確認：

```
gcloud auth application-default print-access-token
```

サービスアカウントの秘密鍵ファイルは作成しない。

4. 必要なAPI

以下のAPIはプロジェクト管理者が有効化する。 すでに有効化済みの場合、再実行は不要。

API	用途
Vertex AI API	Geminiの利用
Cloud Run Admin API	バックエンドのデプロイ
Cloud Firestore API	データベース利用
Cloud Speech-to-Text API	音声認識

有効化コマンド：

```
gcloud services enable aiplatform.googleapis.com run.googleapis.com
firestore.googleapis.com speech.googleapis.com
```

有効化の確認：

```
gcloud services list --enabled
```

APIの有効化にはプロジェクトに対する適切な権限が必要。

5. 環境変数の設定

実値は `.env` に書き、**Gitにコミットしない。**

```
cp .env.example .env
```

`.env.example` の内容：

```
GOOGLE_CLOUD_PROJECT=your-project-id
GOOGLE_CLOUD_LOCATION=asia-northeast1
GEMINI_MODEL=gemini-3.5-flash
```

`.gitignore` に `.env` が含まれていることを確認する。

6. Pythonの準備

```
python3.12 -m venv .venv
source .venv/bin/activate   # Windows: .venv\Scripts\activate
pip install google-genai python-dotenv
```

7. Geminiの疎通確認

`scripts/check_gemini.py` を実行する。

```
python scripts/check_gemini.py
```

Geminiからの返答が表示されれば成功（完了条件）。

```
# scripts/check_gemini.py
import os
from dotenv import load_dotenv
from google import genai

load_dotenv()

client = genai.Client(
    vertexai=True,
    project=os.environ["GOOGLE_CLOUD_PROJECT"],
    location=os.environ["GOOGLE_CLOUD_LOCATION"],
)
resp = client.models.generate_content(
    model=os.environ["GEMINI_MODEL"],
    contents="こんにちは、と一言で返してください。",
)
print(resp.text)
```

8. トラブルシューティング

症状	確認すること
`PERMISSION_DENIED`	IAMロール（Vertex AI ユーザー）が付与されているか / クォータプロジェクトを設定したか
`API has not been used in project`	4章のAPIが有効か（反映に数分かかることがある）
`404 model not found`	モデル名とリージョンの組み合わせが提供されているか
`429 Quota exceeded`	クォータを確認し、必要なら管理者に相談
認証エラー	`gcloud auth application-default login` をやり直す

9. 完了チェックリスト

- [ ] `gcloud auth application-default login` 済み
- [ ] `.env` を作成済み（コミットしていない）
- [ ] `scripts/check_gemini.py` でGeminiの返答を確認

10. 管理者向け手順

> この章はプロジェクト管理者のみが実施する。メンバーは読むだけでよい。

10.1 変数の設定

```
export PROJECT_ID="<プロジェクトID>"
export REGION="asia-northeast1"
```

10.2 Firestoreデータベースの作成

APIを有効化しただけでは使えない。データベースを1つ作成する。

```
gcloud firestore databases create --location=$REGION
```

ロケーションは作成後に変更できないため、事前に確認する。

10.3 バックエンド用サービスアカウントの作成

Cloud Run上のバックエンドが使うサービスアカウントを1つ作る。

```
gcloud iam service-accounts create backend-sa \
  --display-name="Backend service account"
```

10.4 サービスアカウントへのロール付与（最小権限）

ロール	用途
`roles/aiplatform.user`	Vertex AI（Gemini）の呼び出し
`roles/datastore.user`	Firestoreの読み書き
`roles/speech.client`	Speech-to-Textの利用

```
SA="serviceAccount:backend-sa@${PROJECT_ID}.iam.gserviceaccount.com"
for ROLE in roles/aiplatform.user roles/datastore.user roles/speech.client
do
  gcloud projects add-iam-policy-binding $PROJECT_ID \
    --member="$SA" --role="$ROLE"
done
```

オーナーや編集者などの広いロールは付与しない。Cloud Runのデプロイ時に `--service-account` で指定して使う。

10.5 開発メンバーへのIAM付与

メンバーが増減しやすいため、Googleグループに付与するのが望ましい。個人に付与する場合は、メンバーごとに実行する。

ロール	用途
`roles/aiplatform.user`	Geminiの呼び出し
`roles/datastore.user`	Firestoreの読み書き
`roles/run.developer`	Cloud Runのデプロイ
`roles/serviceusage.serviceUsageConsumer`	クォータプロジェクトとしての利用

```
MEMBER="user:<メールアドレス>"
for ROLE in \
  roles/aiplatform.user \
  roles/datastore.user \
  roles/run.developer \
  roles/serviceusage.serviceUsageConsumer
do
  gcloud projects add-iam-policy-binding $PROJECT_ID \
    --member="$MEMBER" --role="$ROLE"
done
```

Cloud Runのデプロイを担当するメンバーには、`backend-sa` に対する `roles/iam.serviceAccountUser` も付与する。

```
gcloud iam service-accounts add-iam-policy-binding \
  backend-sa@${PROJECT_ID}.iam.gserviceaccount.com \
  --member="$MEMBER" --role="roles/iam.serviceAccountUser"
```

10.6 付与結果の確認

```
gcloud projects get-iam-policy $PROJECT_ID \
  --flatten="bindings[].members" \
  --filter="bindings.members:<メールアドレス>" \
  --format="table(bindings.role)"
```

11. 完了条件の確認

Issue #2の完了条件は「メンバー全員がローカルからVertex AIのGeminiを1回呼び出せる」こと。各メンバーが7章の `scripts/check_gemini.py` を実行し、結果を記録する。

メンバー	確認日	結果	備考

失敗した場合は8章のトラブルシューティングを確認する。解決しない場合は、エラーメッセージ全文（トークンや鍵は含めない）を添えて管理者に連絡する。